UROS-26 Investigating Comparative Performance of Numerical Integrators on a Hamiltonian Two-Body System Under Newtonian Mechanics

Mykyta Khomiakov

June 21, 2026

1 ABSTRACT

The celestial dynamics of the N-body problem are most commonly studied through a restricted set of numerical integration methods, often tracking only a limited range of physical variables. Existing literature focuses predominantly on symplectic integrators, including Leapfrog, Velocity Verlet, Symplectic Euler, and the fourth-order Runge-Kutta method. This paper presents a comparative evaluation of five numerical integration schemes: Explicit Euler, Symplectic Euler, Velocity Verlet, Leapfrog, and Runge-Kutta 4th Order with respect to energy conservation, linear and angular momentum conservation, and computational efficiency within a Hamiltonian framework. Building on this comparison, we further investigate a modification of the best-performing integrator, targeting improvements in energy, angular momentum, and orbital stability for the two-body problem. The trade-offs between numerical precision and computational cost introduced by this modification are examined in the context of a high-performance implementation. Our findings have broader implications for the simulation of celestial mechanics across a range of cosmological settings, with particular emphasis on two-body classical gravitational dynamics.

2 INTRODUCTION

When studying N-body celestial systems, selecting an appropriate numerical integration scheme is fundamental to obtaining accurate and meaningful results. Leapfrog and fourth-order Runge-Kutta (RK4) are routinely employed as standard integrators across modern computational astrophysics, particularly in non-chaotic regimes.

This paper investigates the two-body gravitational problem within the frameworks of Newtonian mechanics and Hamiltonian formalism. In classical mechanics, the Hamiltonian is a function describing the total energy

of a system in terms of its generalised coordinates and conjugate momenta, governing the time evolution of the system through Hamilton's equations. This formalism is particularly well-suited to the study of orbital dynamics, as it naturally exposes conserved quantities such as energy and angular momentum against which numerical schemes can be rigorously evaluated.

We restrict our analysis to classical mechanics as defined by Newton and Kepler, applying the governing equations of the field accordingly. A high-performance simulation engine is developed in C++20 to carry out procedurally generated two-body simulations, with the engine exposed to Python 3.12 via a foreign function interface – a common and practical convention in scientific computing workflows. To make the resulting dual-resolution study tractable, simulations are distributed across a fixed-size worker pool, with each worker independently generating a problem instance and executing all five candidate integrators under matching initial conditions. Simulation data is then analysed in Python to draw conclusions regarding integrator accuracy and computational efficiency.

Having identified the most suitable standard integrator, we subsequently construct a higher-order scheme and repeat the analysis under equivalent conditions. Final conclusions are drawn on the basis of energy and angular momentum conservation behaviour, alongside a comparison of computational cost across all tested methods.

3 ANALYTICAL SOLUTIONS

3.1 NORMALISED SYSTEM

Aligning with the goals of this research we will show the analytical solutions for a singular problem setup across five integration schemes under ten timesteps.

Before starting any calculations, it is required to apply the appropriate units of measurement. It was decided to apply a normalised system due to the ease of debugging and interpretation, numerical stability and avoiding the vulnerability of floating-point errors.

Hence why, the units are as follows:

- Distance – Astronomical Units (Earth to Sun Distance)
- Mass – Solar Masses
- Gravitational Attraction – $4\pi^2$
- Timestep – Years

Following studies a timestep of $\Delta t = 10^{-3}\,\mathrm{yr}$ has been selected, since unstable integrators tend to converge later in the simulation producing more

reasonable data. This allows tracking of convergences and energy preservation more accurately.

Due to the assumption of a normalised numerical system the value of Gravitational Attraction Constant (G) has to be derived from those systemic values. Physics remains the same, while the measuring system modifies. Since the normalised system is based on the relationship between the Earth and the Sun, we can derive the value of G that complies with the numerical system.

Applying Newtonian gravity,

$$F = G\frac{m_1 m_2}{r^2}, \tag{1}$$

and the equation for circular motion,

$$F = \frac{m_2 v^2}{r}, \tag{2}$$

we obtain

$$\frac{m_2 v^2}{r} = G\frac{m_1 m_2}{r^2}. \tag{3}$$

Cancelling m_2 from both sides yields

$$v^2 = \frac{Gm_1}{r}. \tag{4}$$

Since the circumference of a circular orbit is $2\pi r$, the orbital velocity can be expressed in terms of the orbital period T as

$$v = \frac{2\pi r}{T}. \tag{5}$$

Substituting this expression for v into Equation (3) gives

$$\left(\frac{2\pi r}{T}\right)^2 = \frac{Gm_1}{r}. \tag{6}$$

Solving for T yields

$$T^2 = \frac{4\pi^2 r^3}{Gm_1}. \tag{7}$$

Assuming a normalised system of units in which the orbital period, orbital radius, and central mass are all equal to unity ($T = r = m_1 = 1$), we obtain

$$1 = \frac{4\pi^2}{G}. \tag{8}$$

Rearranging gives

$$G = 4\pi^2. \tag{9}$$

3.2 ANALYTICAL PROBLEM

The problem definition is as follows: A star of one solar mass $M = 1\,M_\odot$ is fixed at the origin. A planet orbiting the star is 333000^{-1} of solar mass and is placed at initial position $\vec{r}_0 = (2,\,1)^T$ AU. All quantities are expressed in natural astronomical units: distances in AU, time in years, and mass in solar masses, with $G = 4\pi^2\,\mathrm{AU}^3\,\mathrm{yr}^{-2}\,M_\odot^{-1}$. The simulation procedurally generates two objects containing their mass and coordinates, while the rest of the calculations are presented through the later simulation.

3.3 INITIAL VALUES

Initial distance

The initial separation between the planet and the star is

$$r_0 = |\vec{r}_0| = \sqrt{x_0^2 + y_0^2} = \sqrt{2^2 + 1^2} = \sqrt{5} \approx 2.23606798\,\mathrm{AU}. \tag{10}$$

Initial acceleration

Newton's law of gravity gives the acceleration per unit planet mass as

$$\vec{a} = -\frac{GM}{r^3}\vec{r}. \tag{11}$$

Substituting $G = 4\pi^2$, $M = 1$, $r_0 = \sqrt{5}$, $\vec{r}_0 = (2, 1)^T$:

$$\vec{a}_0 = -\frac{4\pi^2}{(\sqrt{5})^3}\begin{pmatrix}2\\1\end{pmatrix} = -\frac{4\pi^2}{5\sqrt{5}}\begin{pmatrix}2\\1\end{pmatrix} \approx \begin{pmatrix}-7.06211403\\-3.53105702\end{pmatrix}\,\mathrm{AU\,yr}^{-2}. \tag{12}$$

The magnitude is $|\vec{a}_0| = 4\pi^2/5 \approx 7.8957\,\mathrm{AU\,yr}^{-2}$.

Circular orbital velocity

For a circular orbit, gravitational acceleration exactly supplies centripetal acceleration:

$$v_c = \sqrt{\frac{GM}{r_0}} = \sqrt{\frac{4\pi^2}{\sqrt{5}}} \approx 4.20181926\,\mathrm{AU\,yr}^{-1}. \tag{13}$$

The velocity is directed tangentially (perpendicular to $\vec{r}_0$, counter-clockwise):

$$\hat{t} = \frac{(-y_0,\,x_0)^T}{r_0} = \frac{1}{\sqrt{5}}\begin{pmatrix}-1\\2\end{pmatrix}, \qquad \vec{v}_0 = v_c\,\hat{t} \approx \begin{pmatrix}-1.87911070\\3.75822140\end{pmatrix}\,\mathrm{AU\,yr}^{-1}. \tag{14}$$

Initial conserved quantities

The total mechanical energy (kinetic plus potential) per unit planet mass is

$$E_0 = \frac{1}{2}m|\vec{v}_0|^2 - \frac{GMm}{r_0} = -\frac{GMm}{2r_0} \approx -2.6509 \times 10^{-5}\, M_\odot\, \mathrm{AU}^2\, \mathrm{yr}^{-2}, \quad (15)$$

and the scalar z-component of angular momentum is

$$L_0 = m(x_0 v_{y,0} - y_0 v_{x,0}) \approx 2.8215 \times 10^{-5}\, M_\odot\, \mathrm{AU}^2\, \mathrm{yr}^{-1}. \quad (16)$$

3.4 FIRST TIMESTEP DERIVATION ($\Delta t = 10^{-3}$ yr)

We adopt a time step $\Delta t = 10^{-3}\,\mathrm{yr}$ and derive the first update for each integrator explicitly. Relative errors in the conserved quantities are defined as $\varepsilon_E = |E_n - E_0|/|E_0|$ and $\varepsilon_L = |L_n - L_0|/|L_0|$.

Explicit Euler

Update rule: $\vec{r}_{n+1} = \vec{r}_n + \Delta t\, \vec{v}_n, \quad \vec{v}_{n+1} = \vec{v}_n + \Delta t\, \vec{a}_n$.

Step $n = 0 \to 1$:

$$\vec{r}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} + 10^{-3} \begin{pmatrix} -1.87911070 \\ 3.75822140 \end{pmatrix} = \begin{pmatrix} 1.99812089 \\ 1.00375822 \end{pmatrix} \mathrm{AU},$$

$$\vec{v}_1 = \begin{pmatrix} -1.87911070 \\ 3.75822140 \end{pmatrix} + 10^{-3} \begin{pmatrix} -7.06211403 \\ -3.53105702 \end{pmatrix} = \begin{pmatrix} -1.88617281 \\ 3.75469034 \end{pmatrix} \mathrm{AU\,yr}^{-1}.$$

Both position and velocity are updated using values from the same timestep, so this method is first-order accurate and does not conserve energy over time.

Symplectic Euler

Update rule: velocity is updated first, then position uses the updated velocity: $\vec{v}_{n+1} = \vec{v}_n + \Delta t\, \vec{a}_n, \quad \vec{r}_{n+1} = \vec{r}_n + \Delta t\, \vec{v}_{n+1}$.

Step $n = 0 \to 1$:

$$\vec{v}_1 = \begin{pmatrix} -1.87911070 \\ 3.75822140 \end{pmatrix} + 10^{-3} \begin{pmatrix} -7.06211403 \\ -3.53105702 \end{pmatrix} = \begin{pmatrix} -1.88617281 \\ 3.75469034 \end{pmatrix} \mathrm{AU\,yr}^{-1},$$

$$\vec{r}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} + 10^{-3} \begin{pmatrix} -1.88617281 \\ 3.75469034 \end{pmatrix} = \begin{pmatrix} 1.99811383 \\ 1.00375469 \end{pmatrix} \mathrm{AU}.$$

Symplectic Euler is also first-order, but it preserves the symplectic structure of Hamiltonian mechanics, resulting in bounded (rather than drifting) energy errors.

Velocity Verlet

Update rule: $\vec{r}_{n+1} = \vec{r}_n + \Delta t\,\vec{v}_n + \frac{1}{2}\Delta t^2\,\vec{a}_n, \quad \vec{v}_{n+1} = \vec{v}_n + \frac{\Delta t}{2}(\vec{a}_n + \vec{a}_{n+1})$.

Step $n = 0 \to 1$:

$$\vec{r}_1 = \begin{pmatrix}2\\1\end{pmatrix} + 10^{-3}\begin{pmatrix}-1.87911070\\3.75822140\end{pmatrix} + \frac{10^{-6}}{2}\begin{pmatrix}-7.06211403\\-3.53105702\end{pmatrix} = \begin{pmatrix}1.99811736\\1.00375646\end{pmatrix}\ \text{AU}.$$

Then $\vec{a}_1$ is recomputed from $\vec{r}_1$, and

$$\vec{v}_1 = \begin{pmatrix}-1.87911070\\3.75822140\end{pmatrix} + \frac{10^{-3}}{2}\left[\begin{pmatrix}-7.06211403\\-3.53105702\end{pmatrix} + \vec{a}_1\right] = \begin{pmatrix}-1.88616949\\3.75468371\end{pmatrix}\ \text{AU}\,\text{yr}^{-1}.$$

Leapfrog (Kick–Drift–Kick)

The Leapfrog integrator stores velocities at half-integer steps. The initialisation half-kick is:

$$\vec{v}_{1/2} = \vec{v}_0 + \frac{\Delta t}{2}\vec{a}_0 = \begin{pmatrix}-1.87911070\\3.75822140\end{pmatrix} + 5\times10^{-4}\begin{pmatrix}-7.06211403\\-3.53105702\end{pmatrix} = \begin{pmatrix}-1.88264677\\3.75645587\end{pmatrix}\ \text{AU}\,\text{yr}^{-1}.$$

Full drift step $n = 0 \to 1$:

$$\vec{r}_1 = \vec{r}_0 + \Delta t\,\vec{v}_{1/2} = \begin{pmatrix}2\\1\end{pmatrix} + 10^{-3}\begin{pmatrix}-1.88264677\\3.75645587\end{pmatrix} = \begin{pmatrix}1.99811736\\1.00375646\end{pmatrix}\ \text{AU}.$$

Velocities at integer steps are recovered as $\vec{v}_n = \vec{v}_{n+1/2} - \frac{\Delta t}{2}\vec{a}_n$, producing results identical to Velocity Verlet to floating-point precision.

RK4 (4th-order Runge–Kutta)

Writing the state vector as $\mathbf{y} = (\vec{r},\, \vec{v})^T$ with $\dot{\mathbf{y}} = (\vec{v},\, \vec{a}(\vec{r}))^T$, four intermediate slopes are:

$$\mathbf{k}_1 = f(\mathbf{y}_n), \quad \mathbf{k}_2 = f\left(\mathbf{y}_n + \tfrac{\Delta t}{2}\mathbf{k}_1\right), \quad \mathbf{k}_3 = f\left(\mathbf{y}_n + \tfrac{\Delta t}{2}\mathbf{k}_2\right), \quad \mathbf{k}_4 = f(\mathbf{y}_n + \Delta t\,\mathbf{k}_3),$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)\,.$$

After step $n = 0 \to 1$:

$$\vec{r}_1 \approx \begin{pmatrix}1.99811736\\1.00375645\end{pmatrix}\ \text{AU}, \qquad \vec{v}_1 \approx \begin{pmatrix}-1.88616949\\3.75468371\end{pmatrix}\ \text{AU}\,\text{yr}^{-1}.$$

RK4 is fourth-order accurate; energy errors at each step are $\mathcal{O}(\Delta t^5)$.

3.5 10-STEP SIMULATION TABLES ($\Delta t = 10^{-3}\,\mathbf{yr}$)

The conserved quantities and their relative errors are defined as

$$E_n = \frac{1}{2}m\big(v_{x,n}^2 + v_{y,n}^2\big) - \frac{GMm}{r_n}, \quad L_n = m(x_n v_{y,n} - y_n v_{x,n})\,, \quad \varepsilon_E = \frac{|E_n - E_0|}{|E_0|}, \quad \varepsilon_L = \frac{|L_n - L_0|}{|L_0|}. \tag{17}$$

Units: positions and distances in AU, velocities in $\mathrm{AU\,yr^{-1}}$, accelerations in $\mathrm{AU\,yr^{-2}}$, energies in $M_\odot\,\mathrm{AU^2\,yr^{-2}}$, angular momenta in $M_\odot\,\mathrm{AU^2\,yr^{-1}}$.

Explicit Euler

n	$\vec{r}$ (AU)	$\vec{v}$ ($\mathrm{AU\,yr^{-1}}$)	$\vec{a}$ ($\mathrm{AU\,yr^{-2}}$)	$\lvert\vec{v}\rvert$	r	ε_E	ε_L
0	(2.00, 1.00)	(−1.88, 3.76)	(−7.06, −3.53)	4.20	2.24	0	0
1	(2.00, 1.00)	(−1.89, 3.75)	(−7.06, −3.54)	4.20	2.24	7.06×10^{-6}	3.53×10^{-6}
2	(2.00, 1.01)	(−1.89, 3.75)	(−7.05, −3.56)	4.20	2.24	1.41×10^{-5}	7.06×10^{-6}
3	(1.99, 1.01)	(−1.90, 3.75)	(−7.04, −3.57)	4.20	2.24	2.12×10^{-5}	1.06×10^{-5}
4	(1.99, 1.02)	(−1.91, 3.74)	(−7.04, −3.58)	4.20	2.24	2.82×10^{-5}	1.41×10^{-5}
5	(1.99, 1.02)	(−1.91, 3.74)	(−7.03, −3.60)	4.20	2.24	3.53×10^{-5}	1.77×10^{-5}
6	(1.99, 1.02)	(−1.92, 3.74)	(−7.02, −3.61)	4.20	2.24	4.24×10^{-5}	2.12×10^{-5}
7	(1.99, 1.03)	(−1.93, 3.73)	(−7.01, −3.62)	4.20	2.24	4.94×10^{-5}	2.47×10^{-5}
8	(1.98, 1.03)	(−1.94, 3.73)	(−7.01, −3.64)	4.20	2.24	5.65×10^{-5}	2.82×10^{-5}
9	(1.98, 1.03)	(−1.94, 3.73)	(−7.00, −3.65)	4.20	2.24	6.36×10^{-5}	3.18×10^{-5}
10	(1.98, 1.04)	(−1.95, 3.72)	(−6.99, −3.66)	4.20	2.24	7.06×10^{-5}	3.53×10^{-5}

Table 1: Explicit Euler, $\Delta t = 10^{-3}\,\mathrm{yr}$. Energy and angular momentum drift monotonically at $\mathcal{O}(\Delta t)$ per step – characteristic of a non-symplectic first-order method.

Symplectic Euler

n	$\vec{r}$ (AU)	$\vec{v}$ ($\mathrm{AU\,yr^{-1}}$)	$\vec{a}$ ($\mathrm{AU\,yr^{-2}}$)	$\lvert\vec{v}\rvert$	r	ε_E	ε_L
0	(2.00, 1.00)	(−1.88, 3.76)	(−7.06, −3.53)	4.20	2.24	0	0
1	(2.00, 1.00)	(−1.89, 3.75)	(−7.06, −3.54)	4.20	2.24	3.10×10^{-12}	~0
2	(2.00, 1.01)	(−1.89, 3.75)	(−7.05, −3.56)	4.20	2.24	1.30×10^{-11}	~0
3	(1.99, 1.01)	(−1.90, 3.75)	(−7.04, −3.57)	4.20	2.24	2.80×10^{-11}	~0
4	(1.99, 1.01)	(−1.91, 3.74)	(−7.04, −3.58)	4.20	2.24	5.00×10^{-11}	~0
5	(1.99, 1.02)	(−1.91, 3.74)	(−7.03, −3.60)	4.20	2.24	7.80×10^{-11}	~0
6	(1.99, 1.02)	(−1.92, 3.74)	(−7.02, −3.61)	4.20	2.24	1.10×10^{-10}	~0
7	(1.99, 1.03)	(−1.93, 3.73)	(−7.02, −3.62)	4.20	2.24	1.50×10^{-10}	~0
8	(1.98, 1.03)	(−1.94, 3.73)	(−7.01, −3.64)	4.20	2.24	2.00×10^{-10}	~0
9	(1.98, 1.03)	(−1.94, 3.73)	(−7.00, −3.65)	4.20	2.24	2.50×10^{-10}	~0
10	(1.98, 1.04)	(−1.95, 3.72)	(−6.99, −3.66)	4.20	2.24	3.10×10^{-10}	~0

Table 2: Symplectic Euler, $\Delta t = 10^{-3}\,\mathrm{yr}$. Angular momentum is conserved to machine precision; energy error is bounded ($\sim 10^{-10}$–10^{-12}) with no secular drift – hallmark of a symplectic method.

Velocity Verlet

n	$\vec{r}$ (AU)	$\vec{v}$ (AU yr^{-1})	$\vec{a}$ (AU yr^{-2})	$\lvert\vec{v}\rvert$	r	ε_E	ε_L
0	(2.00, 1.00)	(−1.88, 3.76)	(−7.06, −3.53)	4.20	2.24	0	0
1	(2.00, 1.00)	(−1.89, 3.75)	(−7.06, −3.54)	4.20	2.24	$\sim 10^{-16}$	~ 0
2	(2.00, 1.01)	(−1.89, 3.75)	(−7.05, −3.56)	4.20	2.24	$\sim 10^{-16}$	~ 0
3	(1.99, 1.01)	(−1.90, 3.75)	(−7.04, −3.57)	4.20	2.24	$\sim 10^{-16}$	~ 0
4	(1.99, 1.02)	(−1.91, 3.74)	(−7.04, −3.58)	4.20	2.24	$\sim 10^{-16}$	~ 0
5	(1.99, 1.02)	(−1.91, 3.74)	(−7.03, −3.60)	4.20	2.24	$\sim 10^{-16}$	~ 0
6	(1.99, 1.02)	(−1.92, 3.74)	(−7.02, −3.61)	4.20	2.24	$\sim 10^{-16}$	~ 0
7	(1.99, 1.03)	(−1.93, 3.73)	(−7.02, −3.62)	4.20	2.24	$\sim 10^{-16}$	~ 0
8	(1.98, 1.03)	(−1.94, 3.73)	(−7.01, −3.64)	4.20	2.24	$\sim 10^{-16}$	~ 0
9	(1.98, 1.03)	(−1.94, 3.73)	(−7.00, −3.65)	4.20	2.24	$\sim 10^{-16}$	~ 0
10	(1.98, 1.04)	(−1.95, 3.72)	(−6.99, −3.66)	4.20	2.24	1.00×10^{-15}	~ 0

Table 3: Velocity Verlet, $\Delta t = 10^{-3}$ yr. Both $|\vec{v}|$ and r remain constant to 3 significant figures throughout; energy errors are at floating-point rounding level ($\sim 10^{-15}$–10^{-16}). This is a second-order symplectic method.

Leapfrog (Kick–Drift–Kick)

n	$\vec{r}$ (AU)	$\vec{v}$ (AU yr^{-1})	$\vec{a}$ (AU yr^{-2})	$\lvert\vec{v}\rvert$	r	ε_E	ε_L
0	(2.00, 1.00)	(−1.88, 3.76)	(−7.06, −3.53)	4.20	2.24	0	0
1	(2.00, 1.00)	(−1.89, 3.75)	(−7.06, −3.54)	4.20	2.24	~ 0	~ 0
2	(2.00, 1.01)	(−1.89, 3.75)	(−7.05, −3.56)	4.20	2.24	$\sim 10^{-16}$	~ 0
3	(1.99, 1.01)	(−1.90, 3.75)	(−7.04, −3.57)	4.20	2.24	$\sim 10^{-16}$	~ 0
4	(1.99, 1.02)	(−1.91, 3.74)	(−7.04, −3.58)	4.20	2.24	$\sim 10^{-16}$	~ 0
5	(1.99, 1.02)	(−1.91, 3.74)	(−7.03, −3.60)	4.20	2.24	$\sim 10^{-16}$	~ 0
6	(1.99, 1.02)	(−1.92, 3.74)	(−7.02, −3.61)	4.20	2.24	$\sim 10^{-16}$	~ 0
7	(1.99, 1.03)	(−1.93, 3.73)	(−7.02, −3.62)	4.20	2.24	$\sim 10^{-16}$	~ 0
8	(1.98, 1.03)	(−1.94, 3.73)	(−7.01, −3.64)	4.20	2.24	$\sim 10^{-16}$	~ 0
9	(1.98, 1.03)	(−1.94, 3.73)	(−7.00, −3.65)	4.20	2.24	$\sim 10^{-16}$	~ 0
10	(1.98, 1.04)	(−1.95, 3.72)	(−6.99, −3.66)	4.20	2.24	$\sim 10^{-16}$	~ 0

Table 4: Leapfrog (KDK), $\Delta t = 10^{-3}$ yr. Results are numerically identical to Velocity Verlet (they are algebraically equivalent for constant-mass conservative systems). The Leapfrog is distinguished by its staggered storage of velocities at half-integer steps.

RK4 (4th-order Runge–Kutta)

n	$\vec{r}$ (AU)	$\vec{v}$ (AU yr^{-1})	$\vec{a}$ (AU yr^{-2})	$\lvert\vec{v}\rvert$	r	ε_E	ε_L
0	(2.00, 1.00)	(−1.88, 3.76)	(−7.06, −3.53)	4.20	2.24	0	0
1	(2.00, 1.00)	(−1.89, 3.75)	(−7.06, −3.54)	4.20	2.24	$\sim$0	$\sim$0
2	(2.00, 1.01)	(−1.89, 3.75)	(−7.05, −3.56)	4.20	2.24	$\sim$0	$\sim$0
3	(1.99, 1.01)	(−1.90, 3.75)	(−7.04, −3.57)	4.20	2.24	$\sim$0	$\sim$0
4	(1.99, 1.02)	(−1.91, 3.74)	(−7.04, −3.58)	4.20	2.24	$\sim$0	$\sim$0
5	(1.99, 1.02)	(−1.91, 3.74)	(−7.03, −3.60)	4.20	2.24	$\sim$0	$\sim$0
6	(1.99, 1.02)	(−1.92, 3.74)	(−7.02, −3.61)	4.20	2.24	$\sim 10^{-16}$	$\sim$0
7	(1.99, 1.03)	(−1.93, 3.73)	(−7.02, −3.62)	4.20	2.24	$\sim 10^{-16}$	$\sim$0
8	(1.98, 1.03)	(−1.94, 3.73)	(−7.01, −3.64)	4.20	2.24	$\sim$0	$\sim$0
9	(1.98, 1.03)	(−1.94, 3.73)	(−7.00, −3.65)	4.20	2.24	$\sim$0	$\sim$0
10	(1.98, 1.04)	(−1.95, 3.72)	(−6.99, −3.66)	4.20	2.24	$\sim$0	$\sim$0

Table 5: RK4, $\Delta t = 10^{-3}$ yr. Both $|\vec{v}|$ and r are constant to 3 significant figures throughout; energy errors are entirely at floating-point rounding level. RK4 is not symplectic but is 4th-order accurate, so local errors are $\mathcal{O}(\Delta t^5)$.

3.6 SUMMARY OF INTEGRATOR PERFORMANCE

Integrator	Order	ε_E at $n = 10$	Conservation behaviour
Explicit Euler	1	7.06×10^{-5}	Monotonic drift (non-symplectic)
Symplectic Euler	1	3.10×10^{-10}	Bounded oscillation (symplectic)
Velocity Verlet	2	$\sim 10^{-15}$	Near machine precision (symplectic)
Leapfrog	2	$\sim 10^{-16}$	Identical to Velocity Verlet
RK4	4	$\sim 10^{-16}$	Near machine precision (not symplectic)

Table 6: Energy relative error ε_E at $n = 10$ for each integrator with $\Delta t = 10^{-3}$ yr.

Key observations:

- **Explicit Euler** loses energy monotonically at rate $\mathcal{O}(\Delta t)$ per step – unacceptable for long-term orbital integration. At the finer $\Delta t = 10^{-3}$, however, the error at $n = 10$ is 7.06×10^{-5}, two orders of magnitude smaller than at $\Delta t = 10^{-2}$.
- **Symplectic Euler** oscillates around the true energy with no secular drift, despite having the same first-order cost. Energy error ($\sim 3 \times 10^{-10}$) is orders of magnitude lower than Explicit Euler; angular momentum is conserved to machine precision.
- **Velocity Verlet** and **Leapfrog** are algebraically equivalent and provide near-machine-precision conservation at second-order cost – the standard choice in N-body simulation. At $\Delta t = 10^{-3}$ the energy error is entirely at the floating-point rounding floor ($\sim 10^{-15}$–10^{-16}).

- **RK4** gives the lowest error per step at this timestep, with energy errors indistinguishable from machine epsilon. However, RK4 is not symplectic, so secular energy drift will appear on very long timescales; it also requires four force evaluations per step (versus one for Euler and two for Verlet).

4 Methods

This investigation follows a structured methodology designed to minimise bias and maximise the accuracy and reproducibility of results. Synthetic data are generated using a custom C++ simulator across a variable number of timesteps and procedurally constructed initial problem configurations.

4.1 Procedural Generation

Initial conditions are procedurally generated using a Mersenne–Twister pseudo-random number engine with fixed seed parameters to ensure reproducibility. Two physical quantities require generation for each system: the mass of the central object and the spatial coordinates of the orbiting body.

Central body mass. The stellar mass is sampled uniformly from the range $[1.0,\ 2.4]\,M_\odot$. This interval is chosen to restrict the simulation domain to main-sequence stars of spectral classes G through A, encompassing objects analogous to the Sun and Vega, respectively. This restriction is physically motivated: the gravitational influence of more massive stars (class O or B) would cause rapid orbital decay or ejection, while sub-solar-mass stars would introduce qualitatively different dynamical regimes outside the scope of this study.

Planetary mass. The mass of each orbiting body is derived from the stellar mass via the fixed ratio

$$m_{\text{planet}} = \frac{M_\star}{333{,}000}, \tag{18}$$

which approximates the observed mass ratio between Earth and the Sun, providing a physically realistic planetary-mass analogue for each generated system.

Orbital coordinates. The initial separation between the central body and the orbiting object is sampled uniformly from the range $[1,\ 5]\,\text{AU}$. This interval is deliberately chosen to prevent premature convergence in the numerical integrator and to ensure a well-conditioned, stable dataset throughout the integration period.

4.2 Simulation

Following procedural generation, the derived initial quantities velocity, acceleration, and separation distance are computed analytically, alongside baseline stability diagnostics for angular momentum and total energy.

4.2.1 Parallel Compute Pipeline

Simulations are executed concurrently via a fixed-size worker pool of four CPU cores, with each worker assigned an independently generated orbital problem subject to the physical constraints described above. For every assigned problem, the worker executes all five candidate integrators – Explicit Euler, Symplectic Euler, Leapfrog, Velocity Verlet, and RK4 – under identical initial conditions, ensuring that comparisons between schemes are made on a like-for-like basis.

Resolution study. For each integrator, two solutions are computed:

- **Baseline resolution:** $N = 10^5$ steps with $\Delta t = 10^{-2}$.
- **Higher resolution:** $N = 10^6$ steps with $\Delta t = 10^{-3}$.

The higher-resolution solution serves as the reference trajectory for convergence analysis, against which the baseline solution is compared. As both regimes share the product $N \cdot \Delta t$, this comparison isolates the effect of step size on integrator accuracy while holding the total integrated time constant.

Data collection. For each simulation run, the raw trajectory is recorded at every timestep, comprising time, position, velocity, total energy, angular momentum, and any additional state variables required for downstream diagnostics. Each run is written to an individual `.csv` file, preserving full trajectory resolution for subsequent analysis. The summary table of each run will be produced, containing information on the ID of each simulation, angular momentum and energy conservation, and compute time for each integrator.

Graph plots of the trajectories will be produced with both timesteps, however the conservation and relative error data will be plotted from the summary table.

Summary statistics. For each simulation, the following scalar metrics are computed and appended to a master summary table: runtime, maximum and mean relative energy error, and maximum and mean relative angular momentum error. This table forms the basis for all cross-integrator comparisons reported in later sections.

Analysis and visualisation. Post-processing and visualisation are performed using `matplotlib` under Python 3.12. Three categories of plot are produced:

1. *Trajectory-level plots* for each integrator: orbital trajectory in the xy-plane, energy conservation over time, angular momentum conservation over time, relative energy error over time, and relative angular momentum error over time.

2. *Summary comparison plots* across integrators: average runtime, maximum and mean relative energy error, and maximum and mean relative angular momentum error.

3. *Convergence plots*: an overlay of trajectories at $\Delta t = 10^{-2}$ and $\Delta t = 10^{-3}$, a comparison of numerical drift between the two resolutions, and a comparison of accuracy versus computational cost across all integrators.

4.3 Integrator Derivation

In line with one of the primary objectives of this research the development and evaluation of novel numerical schemes, the best-performing integrator identified among the five candidates will be extended to a higher order via appropriate composition or correction methods. This derived integrator will then be benchmarked against its lower-order counterpart under identical simulation conditions.

4.4 Hypothesis

Drawing on prior analytical work and the existing literature on geometric numerical integration, the primary hypothesis is that the *Velocity Verlet* integrator will deliver the most favourable trade-off between computational efficiency and energy fidelity. This expectation is grounded in two properties of the method: its symplecticity, which guarantees long-term conservation of the symplectic structure of Hamiltonian systems, and its second-order accuracy, which provides a sufficient resolution for orbital dynamics without the overhead of higher-order schemes.

Integrator performance will be evaluated against three criteria: (i) long-term conservation of total energy, (ii) preservation of angular momentum, and (iii) computational cost, measured in wall-clock time per integration step.